

Tema 4.4: HTML5 Semántico y CSS3 Avanzado (Flexbox, Grid y Mobile-First)

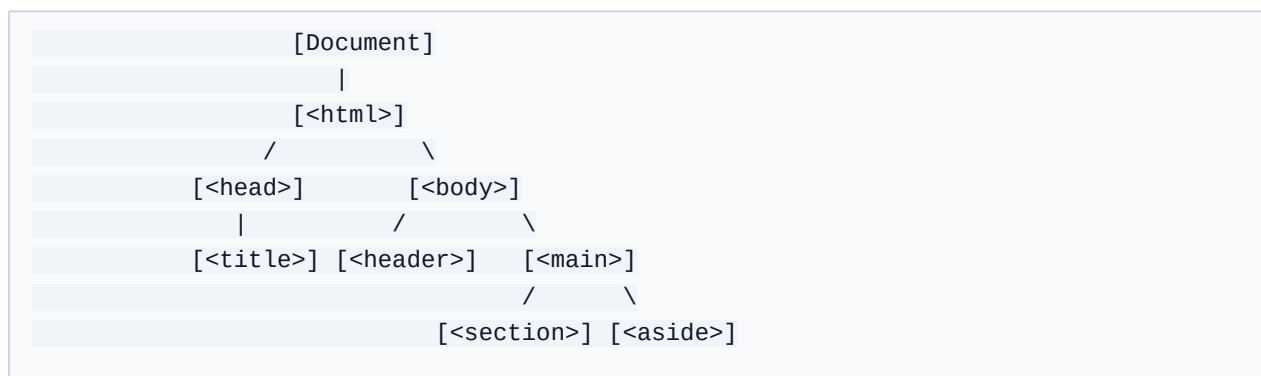
Resumen

Este tema proporciona una inmersión técnica profunda en los lenguajes fundamentales del desarrollo web en el lado del cliente (*Front-End*): **HTML5** y **CSS3**. Se analiza la construcción de estructuras semánticas accesibles a través del Árbol del Modelo de Objetos del Documento (*Document Object Model - DOM*), la gestión de formularios accesibles con validación nativa, y el dominio del **Modelo de Caja (Box Model)** y la especificidad CSS. Además, se abordan las técnicas modernas de diagramación cliente mediante **CSS Flexbox** y **CSS Grid Layout**, la arquitectura de diseño responsivo basado en el principio **Mobile-First** con Media Queries, unidades relativas (`rem`, `em`, `vh`, `vw`, `ch`) y Variables CSS nativas. Incluye una práctica guiada paso a paso para maquetar un panel de control profesional (*Dashboard*) responsivo de alto rendimiento.

Concepto Principal (El 'Qué')

4.4.1 HTML5 Semántico y Estructura del DOM Tree

El lenguaje **HTML5** (*HyperText Markup Language 5*) define la estructura y el significado del contenido web. El navegador web procesa el documento HTML y construye en memoria una estructura jerárquica de nodos denominada el **Árbol del DOM (Document Object Model Tree)**.



Etiquetas Semánticas de Estructura Principales

A diferencia de los contenedores genéricos `<div>` y `<span>`, las etiquetas semánticas de HTML5 aportan significado directo al motor de renderizado, motores de búsqueda (SEO) y tecnologías asistivas (lectores de pantalla):

- `<header>`: Encabezado del documento o de una sección contenedora (logotipos, títulos, barras de navegación primarias).
- `<nav>`: Bloque de navegación que contiene enlaces principales de la aplicación.
- `<main>`: Contenido principal e incremental del cuerpo del documento (`<body>`). Solo debe existir un elemento `<main>` activo por página.
- `<section>`: Agrupación temática de contenidos, típicamente acompañada de un encabezado (`<h2>`-`<h6>`).
- `<article>`: Composición autónoma e independiente que puede ser distribuida de forma individual (entradas de blog, tarjetas de productos, comentarios).
- `<aside>`: Contenido tangencialmente relacionado con el bloque principal (barras laterales, enlaces de referencia, anuncios).
- `<footer>`: Pie de página o final de sección (derechos de autor, avisos legales, datos de contacto).

Creación de Formularios Accesibles en HTML5

Un formulario web debe garantizar la interactividad universal (estándar WCAG 2.1 AA) mediante la vinculación explícita entre etiquetas `<label>` y controles `<input>` mediante el atributo `for`:

```
<form id="frmUsuario" class="form-grid">
  <fieldset>
    <legend>Datos Personales del Cliente</legend>
    <div class="form-group">
      <label for="txtNombre">Nombre Completo:</label>
      <input type="text"
        id="txtNombre"
        name="nombre"
        required
        minlength="3"
        maxlength="50"
      >
```

```

        placeholder="Ej: Ana Maria Lopez"
        autocomplete="name">
    </div>
    <div class="form-group">
        <label for="txtCorreo">Correo Electronico:</label>
        <input type="email"
            id="txtCorreo"
            name="correo"
            required
            placeholder="usuario@dominio.com"
            autocomplete="email">
    </div>
    <button type="submit" class="btn-submit">Guardar Registro</button>
</fieldset>
</form>

```

4.4.2 Hojas de Estilo en Cascada (CSS3) e Internals de Renderizado

CSS3 (Cascading Style Sheets) controla la presentación visual y maquetación del DOM. El motor CSS calcula la apariencia aplicando tres reglas fundamentales: **Cascada**, **Herencia** y **Especificidad**.

El Modelo de Caja (Box Model)

Cada elemento HTML renderizado en la pantalla se calcula como una caja rectangular compuesta por cuatro capas concéntricas:

```

+-----+
| Margin (Margen externo transparente) |
| +-----+ |
| | Border (Borde del elemento) | | | | | |
| | +-----+ | |
| | | Padding (Relleno interno) | | |
| | | +-----+ | | |
| | | | Content (Contenido: texto, | | | |
| | | | imagenes, nodos) | | | |
| | | +-----+ | | |
| | +-----+ | |
| +-----+ |

```

```
+-----+
```

Por defecto, en el modelo tradicional `box-sizing: content-box`, el ancho final de un elemento se calcula como:

$$\text{Ancho Total} = \text{width} + \text{padding-left} + \text{padding-right} + \text{border-left} + \text{border-right}$$

Para evitar desbordamientos inesperados en diagramaciones responsivas, se debe aplicar globalmente la regla moderna:

```
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

Con `box-sizing: border-box`, el ancho declarado (`width`) incluye el `padding` y el `border`.

Especificidad CSS

La especificidad determina qué regla de estilo prevalece cuando múltiples selectores apuntan al mismo elemento. Se representa mediante una tupla de cuatro valores (`a`, `b`, `c`, `d`):

1. **Inline Styles (a):** Estilos aplicados directamente mediante el atributo `style="..."` (Peso: 1000).
2. **ID Selectors (b):** Selectores identificadores únicos `#id` (Peso: 100).
3. **Class, Attribute & Pseudo-classes (c):** Selectores de clase `.clase`, atributos `[type="text"]` y pseudo-clases `:hover`, `:focus`, `:nth-child()` (Peso: 10).
4. **Element & Pseudo-elements (d):** Etiquetas HTML `h1`, `div`, `p` y pseudo-elementos `::before`, `::after` (Peso: 1).

4.4.3 Layouts Modernos: Flexbox y CSS Grid Layout

El estándar CSS3 provee dos motores complementarios de maquetación:

1. CSS Flexbox (Diagramación Unidimensional)

Ideal para distribuir elementos a lo largo de una sola dirección (fila horizontal o columna vertical).

- **Contenedor Flex (`display: flex`):**
- `flex-direction`: Define el eje principal (`row` | `column`).
- `justify-content`: Alinea los elementos en el **eje principal** (`flex-start` | `center` | `space-between` | `space-around`).
- `align-items`: Alinea los elementos en el **eje cruzado** (`stretch` | `center` | `flex-start` | `flex-end`).
- `flex-wrap`: Permite que los elementos pasen a una nueva línea (`nowrap` | `wrap`).

2. CSS Grid Layout (Diagramación Bidimensional)

Diseñado para maquetar interfaces complejas en columnas y filas simultáneas.

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));  
  gap: 1.5rem;  
}
```

- `grid-template-columns`: Define el patrón de columnas.
- `repeat(auto-fit, minmax(280px, 1fr))`: Crea una retícula elástica autoadaptable sin necesidad de escribir múltiples *media queries*. Cada columna tendrá un tamaño mínimo de 280px y ocupará una fracción proporcional (1fr) del espacio sobrante.

4.4.4 Diseño Responsivo, Unidades Relativas y Variables CSS

Unidades Relativas Modernas

- `rem` (*Root EM*): Unidad relativa al tamaño de fuente del elemento raíz `<html>` (por defecto `1rem = 16px`). Permite escalado accesible de texto.
- `em`: Unidad relativa al tamaño de fuente del elemento padre.

- `vh` / `vw`: Porcentaje del alto (`1vh = 1%`) o ancho (`1vw = 1%`) del área visible del navegador (*Viewport*).

Variables CSS (Custom Properties)

Permiten declarar tokens de diseño reutilizables y dinámicos:

```
:root {
  --primary-color: #2563eb;
  --surface-dark: #0f172a;
  --text-main: #f8fafc;
  --radius-card: 12px;
}

.card {
  background-color: var(--surface-dark);
  color: var(--text-main);
  border-radius: var(--radius-card);
}
```

Principio Mobile-First y Media Queries

El enfoque **Mobile-First** consiste en escribir primero los estilos base para dispositivos móviles pequeños y expandir progresivamente la interfaz hacia pantallas de escritorio mediante Media Queries condicionales con `min-width`:

```
/* Estilos Base para Dispositivos Moviles */
.main-layout {
  display: flex;
  flex-direction: column;
}

/* Adaptacion para Pantallas de Escritorio (Tablet/Desktop) */
@media (min-width: 768px) {
  .main-layout {
    display: grid;
    grid-template-columns: 240px 1fr;
  }
}
```

Analogía (La Intuición)

Imagine organizar una exposición de arte en una galería profesional:

- **HTML5 Semántico es la Señalética y Catálogo:** Es la rotulación en la entrada que indica dónde están las esculturas (`<main>`), la guía de navegación en la pared (`<nav>`) y las fichas técnicas que describen cada pintura (`<article>`). Un espectador con discapacidad visual puede usar la audioguía (lector de pantalla) gracias a la correcta descripción del catálogo.
- **El Modelo de Caja (Box Model) son los Marcos de las Pinturas:** El lienzo es el *Content*, el paspartú de cartón blanco alrededor es el *Padding*, el marco de madera tallada es el *Border*, y la distancia requerida entre dos cuadros adyacentes para que no se choquen es el *Margin*.
- **Flexbox es una Fila de Estantes de Exhibición:** Permite acomodar una serie de esculturas de diferentes tamaños en una sola repisa horizontal, ajustando automáticamente la distancia entre ellas para que se vean equilibradas.
- **CSS Grid es el Plano Arquitectónico del Museo:** Divide el salón en una retícula perfecta con pasillos ordenados en filas y columnas, asignando salas de exhibición grandes para obras maestras y pequeñas para bocetos.

Ejemplo de Código e Implementación Paso a Paso (El 'Cómo')

A continuación se desarrolla una **Práctica Guiada Avanzada Paso a Paso** para maquetar un **Panel de Control de Logística y Envíos (Dashboard)** responsivo, utilizando HTML5 Semántico, CSS Variables, Flexbox, CSS Grid y Media Queries Mobile-First.

Paso 1: Estructura HTML5 Semántica y Formulario (`index.html`)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>LogiExpress - Panel de Control</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
  <header class="app-header">
    <div class="logo-area">
      <span class="icon">■</span>
      <h1>LogiExpress Dashboard</h1>
    </div>
    <nav class="user-nav">
      <span>Usuario: <strong>Admin Ops</strong></span>
    </nav>
  </header>

  <div class="dashboard-layout">
    <aside class="sidebar">
      <nav class="sidebar-menu">
        <ul>
          <li><a href="#" class="active">■ Resumen</a></li>
          <li><a href="#">■ Paquetes</a></li>
          <li><a href="#">■ Rutas</a></li>
          <li><a href="#">■■ Ajustes</a></li>
        </ul>
      </nav>
    </aside>

    <main class="content-area">
      <section class="metrics-grid">
        <article class="card-metric">
          <h3>Envíos Activos</h3>
          <p class="metric-value">1,248</p>
        </article>
        <article class="card-metric">
          <h3>En Tránsito</h3>
          <p class="metric-value">856</p>
        </article>
        <article class="card-metric">
          <h3>Entregados Hoy</h3>
          <p class="metric-value">392</p>
        </article>
      </section>

      <section class="panel-section">
        <h2>Registrar Nuevo Paquete</h2>
        <form id="frmPaquete" class="form-grid">
          <div class="field">

```

```

        <label for="txtTracking">Codigo Rastreo:</label>
        <input type="text"
            id="txtTracking"
            name="tracking"
            required
            placeholder="PKG-8849-CR">
    </div>
    <div class="field">
        <label for="numPeso">Peso (kg):</label>
        <input type="number"
            id="numPeso"
            name="peso"
            step="0.1"
            min="0.1"
            max="30.0"
            required
            placeholder="Max 30.0 kg">
    </div>
    <div class="field">
        <label for="cboEstado">Estado Inicial:</label>
        <select id="cboEstado" name="estado" required>
            <option value="">Seleccione estado</option>
            <option value="REGISTRADO">REGISTRADO</option>
            <option value="EN_TRANSITO">EN TRANSITO</option>
        </select>
    </div>
    <div class="field full-width">
        <button type="submit" class="btn-primary">
            Guardar Paquete
        </button>
    </div>
</form>
</section>
</main>
</div>
</body>
</html>

```

Paso 2: Estilos CSS3 Avanzados con Variables, Grid y Flexbox (styles.css)

```

/* ----- */
/* 1. Reset Global y Custom Properties (CSS Variables) */
/* ----- */
:root {
  --bg-main: #0b0f19;
  --bg-surface: #111827;
  --bg-card: #1e293b;
  --primary: #38bdf8;
  --primary-hover: #0284c7;
  --text-primary: #f8fafc;
  --text-secondary: #94a3b8;
  --border-color: #334155;
  --radius: 10px;
  --gap-base: 1.25rem;
}

*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
  background-color: var(--bg-main);
  color: var(--text-primary);
  line-height: 1.5;
}

/* ----- */
/* 2. Header de la Aplicacion (Flexbox Unidimensional) */
/* ----- */
.app-header {
  background-color: var(--bg-surface);
  border-bottom: 1px solid var(--border-color);
  padding: 1rem 1.5rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.logo-area {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

```

```

    .logo-area h1 {
        font-size: 1.25rem;
        color: var(--primary);
    }

/* ----- */
/* 3. Mobile-First Layout y Sidebar Navigation */
/* ----- */

.dashboard-layout {
    display: flex;
    flex-direction: column;
    gap: var(--gap-base);
    padding: var(--gap-base);
}

.sidebar-menu ul {
    list-style: none;
    display: flex;
    gap: 0.5rem;
    overflow-x: auto;
}

.sidebar-menu a {
    color: var(--text-secondary);
    text-decoration: none;
    padding: 0.5rem 1rem;
    border-radius: var(--radius);
    display: block;
    white-space: nowrap;
}

.sidebar-menu a.active, .sidebar-menu a:hover {
    background-color: var(--bg-card);
    color: var(--primary);
}

/* ----- */
/* 4. Reticula de Métricas con CSS Grid Layout Bidimensional */
/* ----- */

.metrics-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: var(--gap-base);
    margin-bottom: var(--gap-base);
}

```

```

    .card-metric {
      background-color: var(--bg-card);
      border: 1px solid var(--border-color);
      border-radius: var(--radius);
      padding: 1.25rem;
    }

    .card-metric h3 {
      font-size: 0.9rem;
      color: var(--text-secondary);
      margin-bottom: 0.5rem;
    }

    .metric-value {
      font-size: 1.8rem;
      font-weight: 700;
      color: var(--primary);
    }

    /* ----- */
    /* 5. Formulario Grid Responsivo */
    /* ----- */

    .panel-section {
      background-color: var(--bg-surface);
      border: 1px solid var(--border-color);
      border-radius: var(--radius);
      padding: 1.5rem;
    }

    .panel-section h2 {
      font-size: 1.1rem;
      margin-bottom: 1rem;
      border-bottom: 1px solid var(--border-color);
      padding-bottom: 0.5rem;
    }

    .form-grid {
      display: grid;
      grid-template-columns: 1fr;
      gap: 1rem;
    }

    .field {
      display: flex;
      flex-direction: column;

```

```

    gap: 0.4rem;
}

.field label {
    font-size: 0.85rem;
    color: var(--text-secondary);
}

.field input, .field select {
    background-color: var(--bg-main);
    border: 1px solid var(--border-color);
    color: var(--text-primary);
    padding: 0.6rem 0.8rem;
    border-radius: 6px;
    font-size: 0.95rem;
}

.field input:focus, .field select:focus {
    outline: 2px solid var(--primary);
}

.btn-primary {
    background-color: var(--primary);
    color: var(--bg-main);
    border: none;
    padding: 0.75rem 1.5rem;
    border-radius: 6px;
    font-weight: 700;
    cursor: pointer;
    width: 100%;
}

.btn-primary:hover {
    background-color: var(--primary-hover);
}

/* ----- */
/* 6. Media Query Adaptativa para Escritorio (Desktop Breakpoint >= 768px) */
/* ----- */
@media (min-width: 768px) {
    .dashboard-layout {
        display: grid;
        grid-template-columns: 200px 1fr;
    }

    .sidebar-menu ul {

```

```
    flex-direction: column;
  }

  .form-grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .full-width {
    grid-column: span 2;
  }

  .btn-primary {
    width: auto;
    justify-self: end;
  }
}
```

Trampas Comunes

Olvidar `box-sizing: border-box`:

- *Error*: Definir anchos porcentuales o con 100% y agregar padding: 15px, provocando que los contenedores se desborden horizontalmente fuera de la pantalla.
- *Solución*: Declarar la regla universal `*, *::before, *::after { box-sizing: border-box; }` al inicio de la hoja de estilos.

No Vincular Etiquetas `<label>` con `for` en Formularios:

- *Error*: Colocar texto de formulario sin etiqueta `<label>` o sin vincular el atributo `for="idInput"`.
- *Solución*: Garantizar que cada control tenga un `<label for="...">` cuyo valor coincida exactamente con el `id` del `<input>`, asegurando accesibilidad A11y.

Uso Excesivo de Media Queries Fijas frente a Retículas Elásticas:

- *Error*: Crear decenas de Media Queries con anchos fijos (`width: 320px`, `width: 480px`, `width: 1024px`) que rompen la interfaz en dispositivos intermedios.

- *Solución:* Utilizar motores bidimensionales elásticos con `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))` que se adaptan automáticamente a cualquier dimensión de pantalla.

Pregunta de Reflexión

Al diseñar una Single-Page Application (SPA) accesible en Angular o React:

¿Cómo influye la sustitución indiscriminada de botones `<button type="button">` y enlaces `<a href="...">` por elementos genéricos `<div onclick="...">` en la accesibilidad web (WCAG 2.1 AA), la navegación por teclado (Tab, Enter, Spacebar) y la experiencia del usuario final?